

AI SOFTWARE DEVELOPMENT INTERNSHIP

EMAN MASOOD (BSSE)

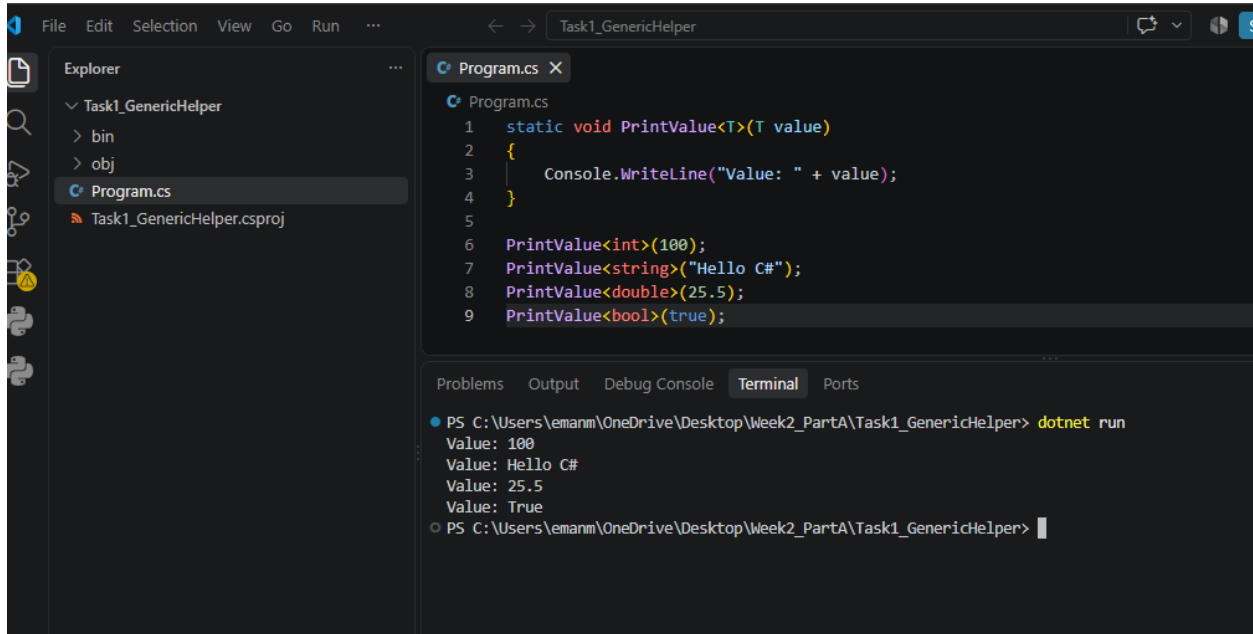
WEEK 2 - Structured C# + ASP.NET Core Basics + Angular Forms

PART A - C# INTERMEDIATE BASICS

Small exercises

1. Create a generic helper method

Output:



The screenshot shows a Visual Studio window with a project named 'Task1_GenericHelper'. The Explorer pane on the left shows the project structure with 'bin', 'obj', 'Program.cs', and 'Task1_GenericHelper.csproj'. The main editor displays 'Program.cs' with the following code:

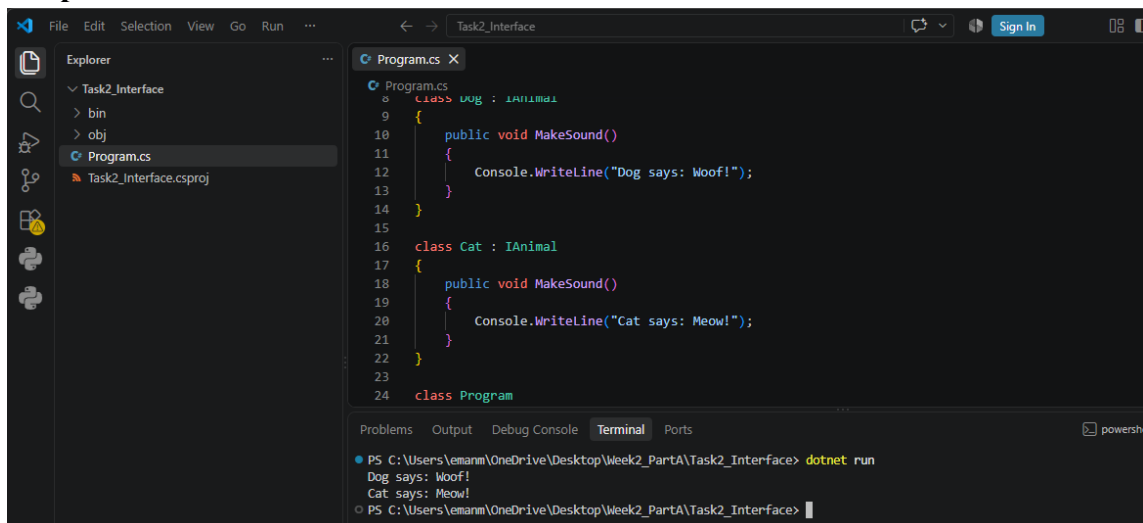
```
1 static void PrintValue<T>(T value)
2 {
3     Console.WriteLine("Value: " + value);
4 }
5
6 PrintValue<int>(100);
7 PrintValue<string>("Hello C#");
8 PrintValue<double>(25.5);
9 PrintValue<bool>(true);
```

The bottom pane shows the 'Terminal' tab with the following output:

```
PS C:\Users\emanm\OneDrive\Desktop\Week2_PartA\Task1_GenericHelper> dotnet run
Value: 100
Value: Hello C#
Value: 25.5
Value: True
PS C:\Users\emanm\OneDrive\Desktop\Week2_PartA\Task1_GenericHelper>
```

2. Create an interface with two implementations

Output:



The screenshot shows a Visual Studio window with a project named 'Task2_Interface'. The Explorer pane on the left shows the project structure with 'bin', 'obj', 'Program.cs', and 'Task2_Interface.csproj'. The main editor displays 'Program.cs' with the following code:

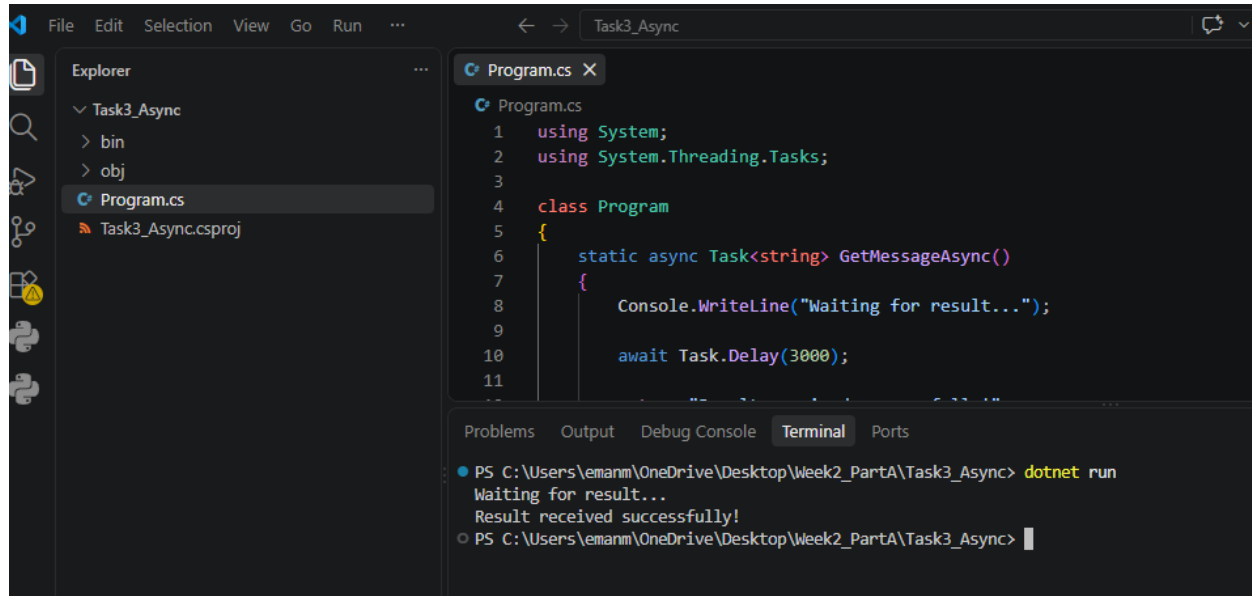
```
9 class IAnimal
10 {
11     public void MakeSound()
12     {
13         Console.WriteLine("Dog says: Woof!");
14     }
15 }
16 class Cat : IAnimal
17 {
18     public void MakeSound()
19     {
20         Console.WriteLine("Cat says: Meow!");
21     }
22 }
23
24 class Program
```

The bottom pane shows the 'Terminal' tab with the following output:

```
PS C:\Users\emanm\OneDrive\Desktop\Week2_PartA\Task2_Interface> dotnet run
Dog says: Woof!
Cat says: Meow!
PS C:\Users\emanm\OneDrive\Desktop\Week2_PartA\Task2_Interface>
```

3. Write an async method that waits and then returns a result

Output:



The screenshot shows the Visual Studio IDE with a project named 'Task3_Async'. The Explorer pane on the left shows the project structure with 'bin', 'obj', 'Program.cs', and 'Task3_Async.csproj'. The main editor displays 'Program.cs' with the following code:

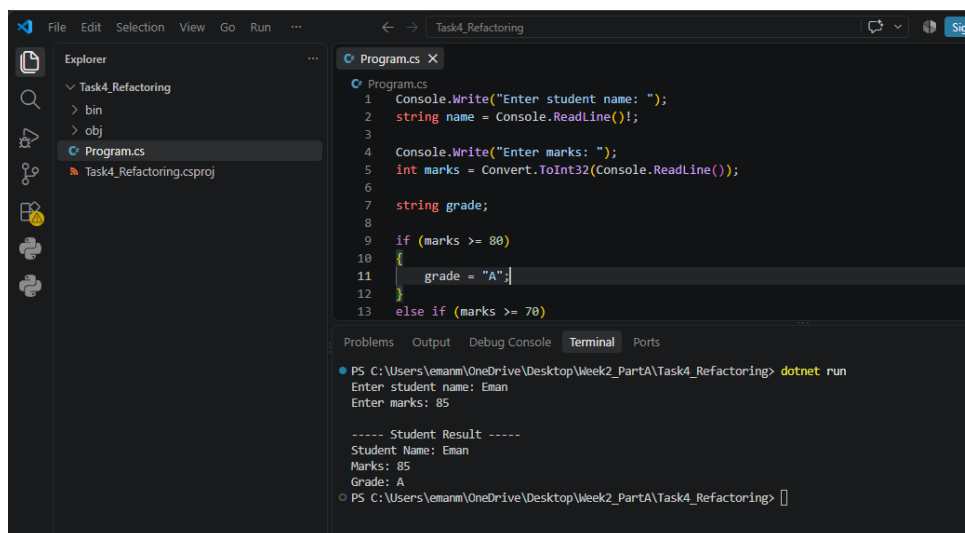
```
1 using System;
2 using System.Threading.Tasks;
3
4 class Program
5 {
6     static async Task<string> GetMessageAsync()
7     {
8         Console.WriteLine("Waiting for result...");
9
10        await Task.Delay(3000);
11    }
12 }
```

The bottom pane shows the 'Terminal' output:

```
PS C:\Users\emamm\OneDrive\Desktop\Week2_PartA\Task3_Async> dotnet run
Waiting for result...
Result received successfully!
PS C:\Users\emamm\OneDrive\Desktop\Week2_PartA\Task3_Async>
```

4. Refactor one messy console program into smaller methods/classes.

Output:



The screenshot shows the Visual Studio IDE with a project named 'Task4_Refactoring'. The Explorer pane on the left shows the project structure with 'bin', 'obj', 'Program.cs', and 'Task4_Refactoring.csproj'. The main editor displays 'Program.cs' with the following code:

```
1 Console.WriteLine("Enter student name: ");
2 string name = Console.ReadLine();
3
4 Console.WriteLine("Enter marks: ");
5 int marks = Convert.ToInt32(Console.ReadLine());
6
7 string grade;
8
9 if (marks >= 80)
10 {
11     grade = "A";
12 }
13 else if (marks >= 70)
```

The bottom pane shows the 'Terminal' output:

```
PS C:\Users\emamm\OneDrive\Desktop\Week2_PartA\Task4_Refactoring> dotnet run
Enter student name: Eman
Enter marks: 85

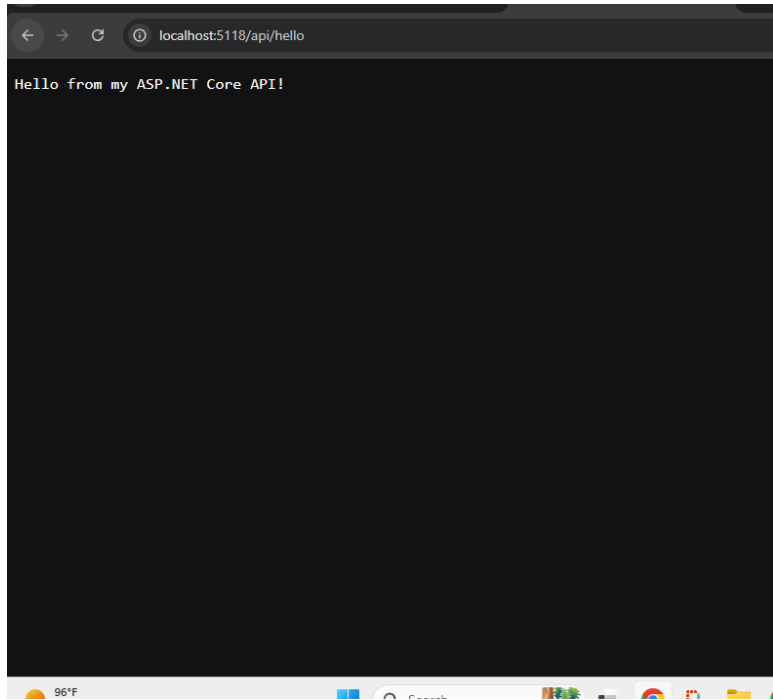
----- Student Result -----
Student Name: Eman
Marks: 85
Grade: A
PS C:\Users\emamm\OneDrive\Desktop\Week2_PartA\Task4_Refactoring>
```

PART B - ASP.NET CORE FUNDAMENTALS

Practice

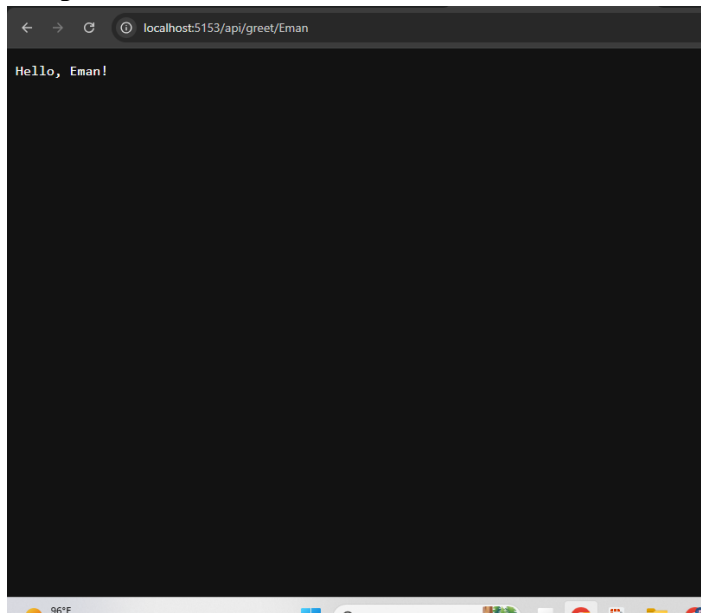
1. Create an ASP.NET Core API with one GET endpoint

Output:



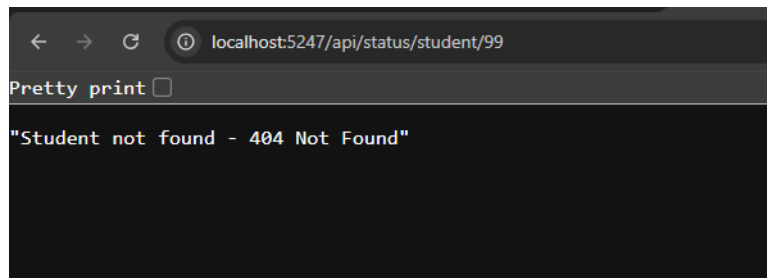
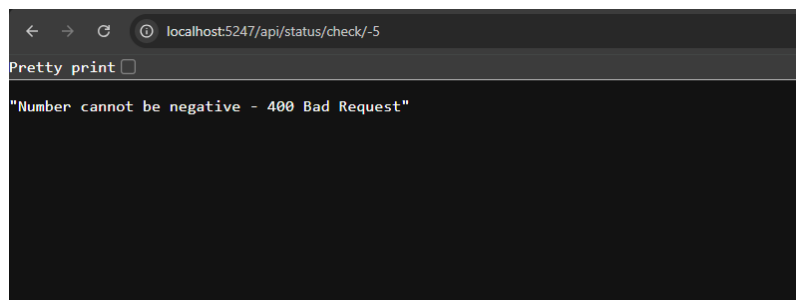
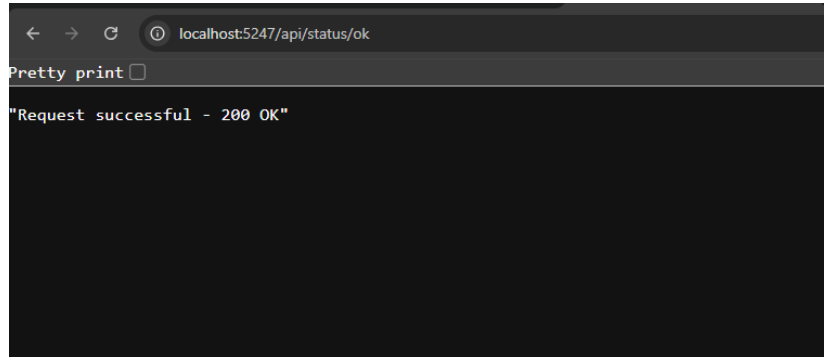
2. Create a second endpoint that accepts input

Output:



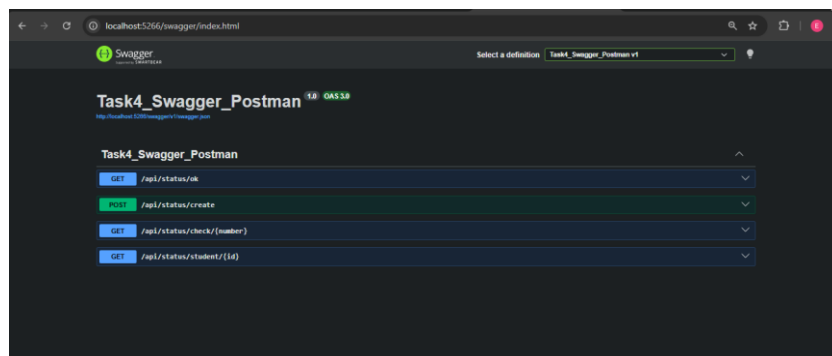
3. Return 200, 201, 400, and 404 in appropriate scenarios

Output:



4. Test every endpoint using Swagger and Postman.

Output:



localhost:5266/swagger/index.html

Swagger
OPENAPI SPECIFICATION

Select a definition Task4_Swagger_Postman v1

Task4_Swagger_Postman 1.0 OAS 3.0

http://localhost:5266/swagger/v1/swagger.json

Task4_Swagger_Postman

GET /api/status/ok

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X GET \
  'http://localhost:5266/api/status/ok' \
  -H 'accept: application/json'
```

Request URL

http://localhost:5266/api/status/ok

Server response

Code Details

200

Response body

POST /api/status/create

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X POST \
  'http://localhost:5266/api/status/create' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

http://localhost:5266/api/status/create

Server response

Code Details

201

Response body

"Data created successfully - 201 Created"

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 28 Aug 2022 11:14:15 GMT
location: /api/status/create
server: Restlet
transfer-encoding: chunked
```

Responses

Code Description Links

localhost:5266/swagger/index.html

Name Description

number * required
integer(int32)
(path)

2

Execute Clear

Responses

Curl

```
curl -X GET \
  'http://localhost:5266/api/status/check/2' \
  -H 'accept: application/json'
```

Request URL

http://localhost:5266/api/status/check/2

Server response

Code Details

200

Response body

"Number is valid - 200 OK"

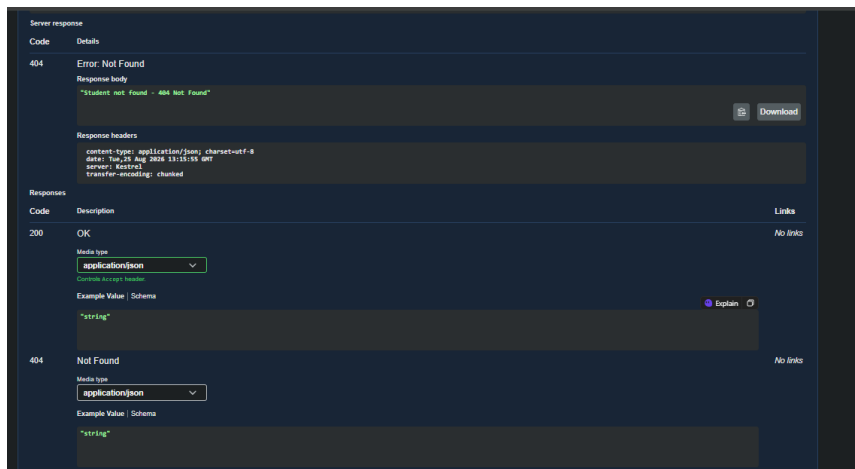
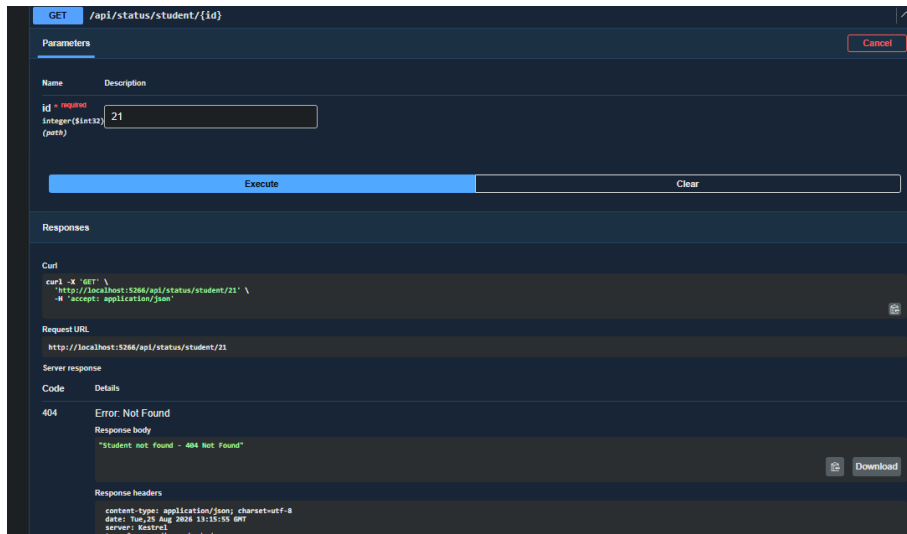
Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 28 Aug 2022 11:14:16 GMT
server: Restlet
transfer-encoding: chunked
```

Responses

Code Description Links

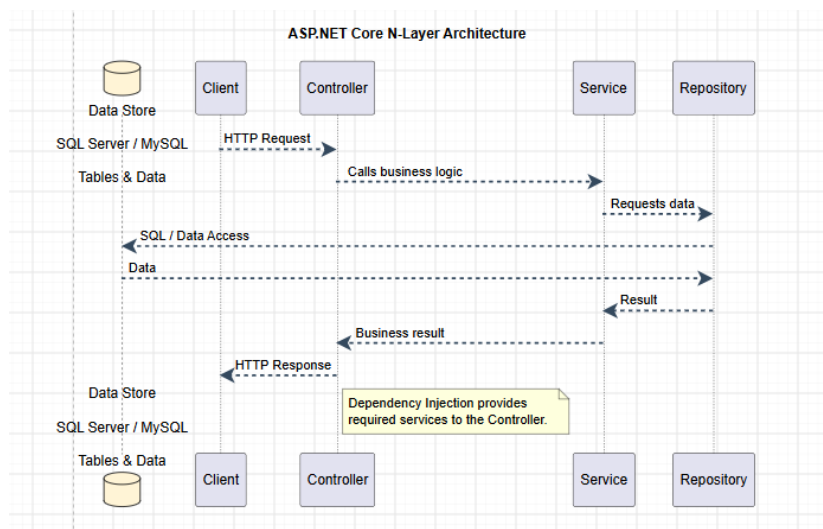
200 OK No links



PART C - COLLECTIONS, EXCEPTIONS, AND LINQ

Practice

1. Draw a simple diagram showing Controller -> Service -> Repository -> Data Store



2. Explain what each layer is responsible for

Presentation / API Layer - Controller

The Presentation or API layer handles communication between the client and the application. The Controller receives HTTP requests such as GET, POST, PUT, and DELETE. It validates basic input and sends the request to the appropriate Service. It then returns the result to the client as an HTTP response. The Controller should not contain complex business or database logic.

Business / Application Layer - Service

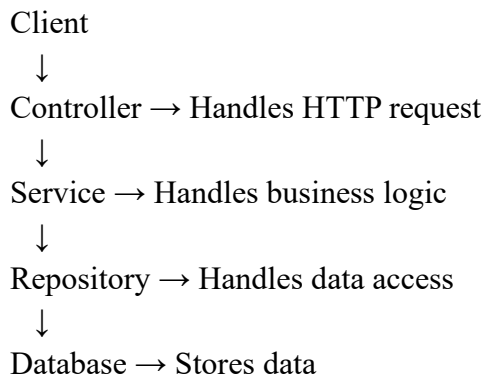
The Business or Application layer contains the main business logic of the application. The Service applies rules, performs calculations, validates business conditions, and decides what operations need to be performed. It receives requests from the Controller and communicates with the Repository when data is required.

Data / Infrastructure Layer - Repository

The Data or Infrastructure layer is responsible for accessing and managing data. The Repository communicates with the database and performs operations such as creating, reading, updating, and deleting records. This keeps database-related code separate from Controllers and business logic.

Data Store - Database

The Data Store is where the application's actual information is stored. It can be a database such as SQL Server or MySQL. It stores records in tables and provides data when requested by the Repository.



3. Find students from a particular department using LINQ

Putting database code directly inside every Controller is a poor design because it makes the application harder to maintain, test, and modify.

For example, if StudentController, TeacherController, and CourseController all contain their own database connection and SQL code, the same type of database logic may be repeated in many places.

This creates several problems:

- **Duplicate code:** The same database operations may be written multiple times.
- **Difficult maintenance:** A database change may require modifying many Controllers.
- **Harder testing:** Controllers become difficult to test because they are directly connected to the database.
- **Poor separation of responsibilities:** Controllers should handle HTTP requests, not database operations.
- **Less flexibility:** Changing the database technology or data-access method becomes more difficult.

A better design is:

Controller



Service



Repository



Database

PART D - ANGULAR BASICS FOR REAL APPLICATIONS

Angular exercises

1. Create a reusable StudentService with mock data.
2. Create a registration form using Reactive Forms.
3. Add required, email, and minimum-length validation.
4. Create two routes and navigate between them

Output:

